WITH CLUSTER
DATA_RETENTION_TIME_IN_DAYS
AUTOINCREMENT

**tables**

```
 CREATE SCD Type 2 Dimension Table
CREATE OR REPLACE TABLE S_CUSTOMER_DIM (
  CUSTOMER_SK          NUMBER AUTOINCREMENT                                  -- surrogate key sequence
, CUSTOMER_ID          VARCHAR(50) COMMENT 'Natural business key'     -- variable-length string
, FIRST_NAME           STRING COMMENT 'First name'                            -- alias for VARCHAR
, LAST_NAME            CHAR(20) COMMENT 'Last name'                           -- fixed-length string
, EMAIL                TEXT COMMENT 'Email address'                           -- unlimited string
, BIRTH_DATE           DATE COMMENT 'Date of birth'                          -- date only
, SIGNUP_TIME          TIMESTAMP_NTZ(9) COMMENT 'Signup timestamp' -- timestamp, no TZ
, LAST_LOGIN_TZ        TIMESTAMP_TZ COMMENT 'Last login w/TZ'         -- timestamp w/ TZ
, PREFERRED_LOCALE     BOOLEAN COMMENT 'Opt-in locale?'                       -- true/false
, CUMULATIVE_POINTS    NUMBER(10,2) COMMENT 'Loyalty points'          -- decimal precision
, PROFILE_PICTURE      BINARY COMMENT 'Avatar image bytes'                    -- binary data
, PREFERENCES          VARIANT COMMENT 'JSON preferences'                     -- semi-structured JSON
, LOCATION_GEOG        GEOGRAPHY COMMENT 'User location point'        -- geospatial data
, VECTOR_EMBEDDING     VECTOR(1536) COMMENT 'AI embedding'           -- vector type for ML
, ACCOUNT_STATUS       OBJECT COMMENT 'JSON key/value status'         -- JSON object variant
, ACTIVE_FROM          TIMESTAMP_LTZ COMMENT 'Record valid from'    -- timestamp local TZ
, ACTIVE_TO            TIMESTAMP_LTZ COMMENT 'Record valid to'        -- SCD end date
, IS_CURRENT           BOOLEAN DEFAULT TRUE COMMENT 'Active flag'    -- current record flag
, RECORD_SOURCE        STRING COMMENT 'Data source system'            -- source identifier
, INGESTION_TIMESTAMP    TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP    COMMENT 'Load time'
                                                                                                  -- auto-timestamp
)
CLUSTER BY (CUSTOMER_ID)
DATA_RETENTION_TIME_IN_DAYS = 14
COMMENT = 'SCD Type 2 Customer Dimension with all Snowflake data types';
```

**Table Information**

```
 2 SHOW & DESC
SHOW TABLES LIKE 'S_CUSTOMER_DIM';
DESC TABLE S_CUSTOMER_DIM;

-- 3 INFORMATION_SCHEMA metadata
SELECT  column_name,  data_type,  comment FROM   INFORMATION_SCHEMA.COLUMNS
WHERE   table_name = 'S_CUSTOMER_DIM';

-- 4 ACCOUNT_USAGE metadata (account-wide)
SELECT   table_catalog,  table_schema,  table_name,  created,  last_altered FROM
SNOWFLAKE.ACCOUNT_USAGE.TABLES
WHERE   table_name = 'S_CUSTOMER_DIM';
```

**TEMPORARY TABLE**

```
CREATE TEMPORARY TABLE temp_customer_activity (
  customer_id    STRING,
  activity_date  DATE,
  activity_type  STRING,
  details        VARIANT
)
COMMENT = 'Session-only table for ad-hoc analysis';

INSERT INTO temp_customer_activity
SELECT id, current_date(), 'login', PARSE_JSON('{"ip":"1.2.3.4"}')
FROM users
WHERE last_login = CURRENT_DATE();

SELECT * FROM temp_customer_activity;

SHOW TABLES LIKE 'TEMP_%';
DESCRIBE TABLE temp_customer_activity;

SELECT
  TABLE_CATALOG,  TABLE_SCHEMA,  TABLE_NAME,  TABLE_TYPE
FROM   INFORMATION_SCHEMA.TABLES
WHERE   TABLE_TYPE = 'TEMPORARY TABLE';

SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.TABLES
WHERE table_type = 'TEMPORARY TABLE';

-- Query as of 5 minutes ago
SELECT * FROM temp_customer_activity
  AT (OFFSET => -60*5);

-- Clone within session
CREATE TEMPORARY TABLE temp_customer_activity_clone CLONE temp_customer_activity;
```

**TRANSIANT TABLE**

```
CREATE OR REPLACE TRANSIENT TABLE my_transient_table (
  id          INT AUTOINCREMENT,
  name        VARCHAR(100) COMMENT 'Entity name',
  created_at    TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP
)
CLUSTER BY (created_at)                -- clustering for pruning:contentReference[oaicite:3]{index=3}
DATA_RETENTION_TIME_IN_DAYS = 1                -- Time Travel retention:contentReference[oaicite:4]{index=4}
COMMENT = 'Sample transient table for staging data';

SHOW TABLES IN SCHEMA my_db.my_schema LIKE '%';

DESC TABLE my_db.my_schema.my_transient_table;

SELECT
  TABLE_CATALOG AS DATABASE_NAME,  TABLE_SCHEMA,  TABLE_NAME,  TABLE_TYPE,  IS_TRANSIENT
FROM my_db.INFORMATION_SCHEMA.TABLES
WHERE IS_TRANSIENT = 'YES';

SELECT
  TABLE_CATALOG,  TABLE_SCHEMA,  TABLE_NAME,  ACTIVE_BYTES,  TIME_TRAVEL_BYTES,  FAILSAFE_BYTES
FROM my_db.INFORMATION_SCHEMA.TABLE_STORAGE_METRICS
WHERE IS_TRANSIENT = 'YES'  AND TABLE_NAME = 'MY_TRANSIENT_TABLE';

-- Zero-copy clone at current state
CREATE TRANSIENT TABLE clone_tr AS
  CLONE my_transient_table;

-- Query as of 10 minutes ago (Time Travel)
SELECT * FROM my_transient_table
  AT (OFFSET => -60*10);

-- Drop transient table
DROP TABLE IF EXISTS my_transient_table;
```

**schemas**

```
-- Create a new schema in a database
CREATE SCHEMA IF NOT EXISTS my_database.my_schema
  COMMENT = 'Staging schema for raw data';

-- Alter schema comment
ALTER SCHEMA my_database.my_schema
  SET COMMENT = 'Updated staging schema note';

-- Drop the schema (must be empty)
DROP SCHEMA IF EXISTS my_database.my_schema;  -- within Time Travel window, can be undropped
:contentReference[oaicite:7]{index=7}

-- List schemas in current or specified database
SHOW SCHEMAS IN DATABASE my_database;  -- includes dropped schemas within retention
:contentReference[oaicite:8]{index=8}

-- Describe schema (lists objects inside)
DESCRIBE SCHEMA my_database.my_schema;  -- shows tables, views, etc.
:contentReference[oaicite:9]{index=9}


SELECT
  catalog_name   AS database_name,  schema_name,  created,  schema_owner
FROM my_database.INFORMATION_SCHEMA.SCHEMATA
ORDER BY schema_name;  -- all schemas in the database :contentReference[oaicite:10]{index=10}


SELECT
  catalog_name   AS database_name,  schema_name,  created_on,  is_transient
FROM SNOWFLAKE.ACCOUNT_USAGE.SCHEMATA
ORDER BY database_name, schema_name;  -- account-wide schema audit
:contentReference[oaicite:11]{index=11}
```

**databases**

```
-- Create a new database (if it doesn't exist)
CREATE OR REPLACE DATABASE my_database
  DATA_RETENTION_TIME_IN_DAYS = 1   -- Time Travel retention (min 1 day)
:contentReference[oaicite:0]{index=0}
  COMMENT = 'Sales analytics database';

-- Drop the database
DROP DATABASE IF EXISTS my_database;  -- objects undroppable only within retention window
:contentReference[oaicite:2]{index=2}

-- List all databases you can access
SHOW DATABASES;  -- includes dropped DBs within Time Travel window
:contentReference[oaicite:3]{index=3}

-- Describe specific database
DESCRIBE DATABASE my_database;
-- shows created, owner, retention, comment :contentReference[oaicite:4]{index=4}

SELECT
  catalog_name    AS database_name,  created,  owner, comment
FROM INFORMATION_SCHEMA.DATABASES ORDER BY database_name;
-- list all databases in current account :contentReference[oaicite:5]{index=5}

SELECT
  database_name,  created_on,  last_altered
FROM SNOWFLAKE.ACCOUNT_USAGE.DATABASES ORDER BY database_name;
-- account-wide history & audit :contentReference[oaicite:6]{index=6}
```

| VIEW | SECURE VIEW | MATERILIZED VIEW |
|---|---|---|
| CREATE OR REPLACE VIEW my_schema.vw_customer_summary AS<br>SELECT customer_id,COUNT(*) AS order_count,SUM(amount)AS total_spent FROM my_schema.orders<br>GROUP BY customer_id;<br><br>ALTER VIEW my_schema.vw_customer_summary RENAME TO vw_cust_summary_v2;<br><br>DROP VIEW IF EXISTS my_schema.vw_customer_summary;<br><br>SHOW VIEWS IN SCHEMA my_schema;<br>DESC VIEW my_schema.vw_customer_summary;<br><br>-- Per-database<br>SELECT table_name, view_definition FROM my_db.INFORMATION_SCHEMA.VIEWS<br>WHERE table_schema = 'MY_SCHEMA';<br><br>-- Account-wide<br>SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.VIEWS<br>WHERE view_schema = 'MY_SCHEMA'; | CREATE SECURE VIEW my_schema.sec_vw_orders AS<br>SELECT * FROM my_schema.orders WHERE region = 'EMEA';<br><br>-- standerd view to secure view<br>ALTER VIEW my_schema.vw_customer_summary SET SECURE;<br><br>-- remove secure property<br>ALTER VIEW my_schema.sec_vw_orders UNSET SECURE;<br><br>SHOW VIEWS IN SCHEMA my_schema LIKE 'SEC_%';<br>DESC VIEW my_schema.sec_vw_orders;<br><br>SELECT table_name, is_secure FROM my_db.INFORMATION_SCHEMA.VIEWS<br>WHERE is_secure = 'YES';<br><br>SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.VIEWS<br>WHERE secure = TRUE; | CREATE SECURE VIEW my_schema.sec_vw_orders AS<br>SELECT * FROM my_schema.orders WHERE region = 'EMEA';<br><br>ALTER VIEW my_schema.vw_customer_summary SET SECURE; -- standerd view to secure view<br><br>ALTER VIEW my_schema.sec_vw_orders UNSET SECURE; -- remove secure property<br><br>SHOW VIEWS IN SCHEMA my_schema LIKE 'SEC_%';<br>DESC VIEW my_schema.sec_vw_orders;<br><br>SELECT table_name, is_secure FROM my_db.INFORMATION_SCHEMA.VIEWS WHERE is_secure = 'YES';<br><br>SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.VIEWS WHERE secure = TRUE;<br><br>-- Refresh History<br>CREATE MATERIALIZED VIEW my_schema.mv_order_totals AS<br>SELECT order_date, SUM(amount) AS daily_total FROM my_schema.orders<br>GROUP BY order_date<br>CLUSTER BY(order_date);<br><br>CREATE SECURE MATERIALIZED VIEW my_schema.sec_mv_orders AS<br>SELECT * FROM my_schema.orders WHERE status = 'CLOSED';<br><br>ALTER MATERIALIZED VIEW my_schema.mv_order_totals SET SECURE -- set secure view<br><br>DROP MATERIALIZED VIEW IF EXISTS my_schema.mv_order_totals;<br><br>SHOW MATERIALIZED VIEWS IN SCHEMA my_schema;<br>DESC MATERIALIZED VIEW my_schema.mv_order_totals;<br><br>SELECT * FROM TABLE (MATERIALIZED_VIEW_REFRESH_HISTORY(<br>'MY_DB.MY_SCHEMA.MV_ORDER_TOTALS',<br>DATEADD('day', -7, CURRENT_TIMESTAMP()),<br>CURRENT_TIMESTAMP()<br>)); |

stages

| internal stage with different parameter | internal stage core syntax | NAME_INTERNAL_STAGE |
|---|---|---|
| Simple Internal Stage with Encryption<br>CREATE OR REPLACE STAGE my_int_stage<br>ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE')<br>COMMENT = 'Encrypted internal stage';<br><br>Temporary Stage with a Default File Format<br>CREATE TEMPORARY STAGE temp_csv_stage<br>FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1 FIELD_OPTIONALLY_ENCLOSED_BY = '"')<br>COMMENT = 'Session-only CSV staging';<br><br>Stage with Directory Table for Fast LIST<br>CREATE OR REPLACE STAGE dir_stage<br>DIRECTORY = (ENABLE = TRUE REFRESH_ON_CREATE = TRUE)<br>FILE_FORMAT = my_parquet_fmt;<br><br>Error-Handling (at COPY TIME) While CREATE STAGE does not take copy options, you handle errors when loading data:<br>COPY INTO target_table<br>FROM @my_int_stage/path/<br>FILE_FORMAT = (FORMAT_NAME = 'my_csv_fmt')<br>ON_ERROR = 'CONTINUE' -- skip bad records<br>VALIDATION_MODE = 'RETURN_ERRORS' -- capture errors without loading<br>PURGE = TRUE; -- optionally remove staged files on success | Core create syntax:<br><br>CREATE [ OR REPLACE ]<br>[ { TEMP \| TEMPORARY } ]<br>STAGE [ IF NOT EXISTS ] <stage_name><br>— Internal-only parameters ————————————<br>[ ENCRYPTION = ( TYPE = 'SNOWFLAKE_FULL'<br>\| TYPE = 'SNOWFLAKE_SSE' ) ]<br>[ DIRECTORY = ( ENABLE = TRUE<br>\| DISABLE = FALSE ) ]<br><br>— Common optional parameters ————————————<br>[ FILE_FORMAT = ( FORMAT_NAME = '<fmt_name>'<br>\| TYPE = { CSV \| JSON \| AVRO \| ORC \| PARQUET \| XML \| CUSTOM }<br>[ <formatTypeOptions> ] ) ]<br>[ COMMENT = '<string>' ]<br>[ [ WITH ] TAG ( <tag1> = '<val1>' [ , <tag2> = '<val2>' ] ) ]<br><br>— Directory table options ————————————<br>[ DIRECTORY = ( ENABLE = TRUE<br>\| REFRESH_ON_CREATE = TRUE<br>\| REFRESH_ON_ACCESS = TRUE ) ]; | CREATE OR REPLACE STAGE my_stage<br>FILE_FORMAT = (TYPE = 'CSV' SKIP_HEADER = 1 FIELD_DELIMITER = ',')<br>COMMENT = 'Customer CSV staging';<br><br>SHOW STAGES IN SCHEMA MY_DB.MY_SCHEMA;<br>DESCRIBE STAGE my_stage;<br><br>LIST @my_stage;<br>LIST @my_stage PATTERN='.*\\.csv';<br><br>PUT file://C:/data/customers.csv @my_stage AUTO_COMPRESS=TRUE;<br>GET @my_stage/customers.csv file://C:/Downloads/;<br><br>COPY INTO CUSTOMER_DIM<br>FROM @my_stage<br>FILE_FORMAT = (FORMAT_NAME = 'MY_CSV_FORMAT')<br>ON_ERROR = 'CONTINUE'; |

| TABLE_STAGE | USER_STAGE | EXTERNAL_STAGE | External_Table |
|---|---|---|---|
| -- List files in the table stage (requires OWNERSHIP on table)<br>LIST @%MY_TABLE;<br><br>-- Upload to the table's stage<br>PUT file://C:/data/orders.json @%MY_TABLE AUTO_COMPRESS=TRUE;<br><br>-- Load staged data directly into the table<br>COPY INTO MY_TABLE<br>FROM @%MY_TABLE<br>FILE_FORMAT = (TYPE = 'JSON')<br>ON_ERROR = 'CONTINUE'; | -- List files in your user stage<br>LIST @~;<br><br>-- Upload local files to your user stage<br>PUT file://C:/data/file.csv @~ AUTO_COMPRESS=TRUE;<br><br>-- Download staged files to local<br>GET @~/file.csv 'file://C:/Downloads/';<br><br>COPY INTO my_table<br>FROM @~<br>FILE_FORMAT = (TYPE = 'CSV' FIELD_OPTIONALLY_ENCLOSED_BY='"')<br>ON_ERROR = 'CONTINUE'; | CREATE OR REPLACE STAGE ext_stage<br>URL = 's3://mybucket/data/'<br>STORAGE_INTEGRATION = my_s3_integration<br>FILE_FORMAT = (TYPE = 'PARQUET')<br>COMMENT = 'External S3 data';<br><br>SHOW STAGES;<br>DESCRIBE STAGE ext_stage;<br><br>LIST @ext_stage;<br>LIST @ext_stage PATTERN='.*\\.parquet';<br><br>COPY INTO ANALYTICS_TABLE<br>FROM @ext_stage<br>FILE_FORMAT = (TYPE = 'PARQUET')<br>ON_ERROR = 'CONTINUE';<br><br>DROP STAGE IF EXISTS ext_stage; | CREATE OR REPLACE EXTERNAL TABLE ext_orders (<br>order_id STRING,<br>amount FLOAT,<br>order_date DATE<br>)<br>WITH LOCATION = @ext_stage<br>FILE_FORMAT = (TYPE = 'PARQUET')<br>AUTO_REFRESH = TRUE;<br><br>ALTER EXTERNAL TABLE ext_orders REFRESH;<br><br>SHOW EXTERNAL TABLES IN SCHEMA MY_DB.MY_SCHEMA;<br>DESCRIBE EXTERNAL TABLE ext_orders;<br><br>DROP EXTERNAL TABLE IF EXISTS ext_orders; |

File format

| CREATE FILE FORMAT Syntax | CSV Format Options | JSON Format Options |
|---|---|---|
| CREATE [ OR REPLACE ] FILE FORMAT <fmt_name><br>TYPE = { CSV \| JSON \| AVRO \| ORC \| PARQUET \| XML }<br>[ FORMAT_TYPE_OPTIONS... ]<br>[ COMPRESSION = { AUTO \| GZIP \| BZ2 \| BROTLI \| ZSTD \| DEFLATE } ]<br>[ COMMENT = '<string>' ]; | CREATE OR REPLACE FILE FORMAT csv_fmt<br>TYPE = CSV<br>FIELD_DELIMITER = ',' -- default: ','<br>SKIP_HEADER = 0 -- default: 0<br>FIELD_OPTIONALLY_ENCLOSED_BY = '"' -- default: none<br>ESCAPE = NONE -- or '\'<br>NULL_IF = ('NULL','') -- default: none<br>TRIM_SPACE = FALSE -- default: FALSE<br>ERROR_ON_COLUMN_COUNT_MISMATCH = FALSE -- default: FALSE<br>DATE_FORMAT = 'AUTO' -- default: AUTO<br>TIME_FORMAT = 'AUTO' -- default: AUTO<br>TIMESTAMP_FORMAT = 'AUTO' -- default: AUTO<br>BINARY_FORMAT = 'HEX' -- default: HEX<br>COMPRESSION = AUTO; -- default: AUTO | CREATE OR REPLACE FILE FORMAT json_fmt<br>TYPE = JSON<br>STRIP_OUTER_ARRAY = FALSE -- treat top-level array<br>IGNORE_UTF8_ERRORS = FALSE -- default: FALSE<br>ALLOW_DUPLICATE -- allow dup keys?<br>KEY => FALSE -- default: FALSE<br>COMPRESSION = AUTO; |

| AVRO, ORC, PARQUET, XML (Semi-Structured) | COPY INTO | Column1 |
|---|---|---|
| CREATE FILE FORMAT avro_fmt<br>TYPE = AVRO<br>COMPRESSION = AUTO; -- auto-detect :contentReference[oaicite:4]{index=4}<br><br>CREATE FILE FORMAT orc_fmt<br>TYPE = ORC<br>COMPRESSION = AUTO;<br><br>CREATE FILE FORMAT parquet_fmt<br>TYPE = PARQUET<br>COMPRESSION = AUTO;<br><br>CREATE FILE FORMAT xml_fmt<br>TYPE = XML<br>STRIP_OUTER_ELEMENT = TRUE -- default: FALSE<br>ENCODING = 'UTF8' -- default: UTF8<br>COMPRESSION = AUTO; | COPY INTO my_table<br>FROM @my_stage<br>FILE_FORMAT = (<br>TYPE = CSV<br>FIELD_DELIMITER = '\|'<br>SKIP_HEADER = 1<br>NULL_IF = ('\\N')<br>)<br>ON_ERROR = 'CONTINUE'<br>VALIDATION_MODE = 'RETURN_ERRORS';<br><br>--Error Handling Options (COPY INTO):<br>COPY INTO my_table<br>FROM @my_stage<br>FILE_FORMAT = (FORMAT_NAME = 'csv_fmt')<br>ON_ERROR = { ABORT_STATEMENT -- default<br>\| CONTINUE<br>\| SKIP_FILE<br>\| SKIP_FILE_<n> }<br>VALIDATION_MODE = { RETURN_ERRORS -- capture and load none<br>\| RETURN_ALL_ERRORS }<br>PURGE = TRUE; -- remove files on success | 2--H16Named Format Usage<br>CREATE FILE FORMAT my_json_fmt TYPE = JSON IGNORE_UTF8_ERRORS = TRUE;<br>COPY INTO events_tbl<br>FROM @json_stage<br>FILE_FORMAT = (FORMAT_NAME = 'my_json_fmt')<br>ON_ERROR = 'SKIP_FILE';<br><br>--Inline Parquet Load<br>COPY INTO parquet_table<br>FROM @pqt_stage<br>FILE_FORMAT = (<br>TYPE = PARQUET<br>COMPRESSION = 'BROTLI'<br>)<br>ON_ERROR = 'CONTINUE'<br>VALIDATION_MODE = 'RETURN_ALL_ERRORS'; |

Pipe

| Common Pipe Operation | AWS S3 Integration & Pipe | Azure Blob Storage Integration & Pipe |
|---|---|---|
| a-- Show all pipes in current schema<br>SHOW PIPES;<br><br>-- Describe a specific pipe<br>DESC PIPE my_db.my_schema.my_pipe;<br><br>-- Drop a pipe<br>DROP PIPE IF EXISTS my_db.my_schema.my_pipe;<br><br>-- Alter a pipe to disable auto-ingest<br>ALTER PIPE my_pipe SET AUTO_INGEST = FALSE;<br><br>-- Refresh a pipe's metadata channel (recreate subscription)<br>ALTER PIPE my_pipe REFRESH; | 1. Create Storage Integration<br>CREATE OR REPLACE STORAGE INTEGRATION aws_int<br>TYPE = EXTERNAL_STAGE<br>STORAGE_PROVIDER = S3<br>ENABLED = TRUE<br>STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::123456789012:role/snowpipe-role'<br>STORAGE_ALLOWED_LOCATIONS = ('s3://my-bucket/path/')<br>COMMENT = 'AWS S3 integration for Snowpipe';<br><br>2 Create Pipe with SNS Topic<br>CREATE OR REPLACE PIPE my_aws_pipe<br>AUTO_INGEST = TRUE<br>AWS_SNS_TOPIC = 'arn:aws:sns:us-west-2:001234567890:my_s3_notifications'<br>AS<br>COPY INTO my_table<br>FROM @my_stage<br>FILE_FORMAT = (TYPE = 'JSON');<br><br>3 SQS Subscription Policy (in Snowflake)<br>SELECT SYSTEM$GET_AWS_SNS_IAM_POLICY(<br>'arn:aws:sns:us-west-2:001234567890:my_s3_notifications'<br>); | 1. Create Storage Integration<br>CREATE OR REPLACE STORAGE INTEGRATION azure_int<br>TYPE = EXTERNAL_STAGE<br>STORAGE_PROVIDER = AZURE<br>ENABLED = TRUE<br>STORAGE_ACCOUNT = 'myaccount.blob.core.windows.net'<br>STORAGE_AZURE_TENANT_ID = 'xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx'<br>STORAGE_AZURE_CLIENT_ID = 'yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyy'<br>STORAGE_AZURE_CLIENT_SECRET = 'your-secret'<br>STORAGE_ALLOWED_LOCATIONS = ('azure://myaccount.blob.core.windows.net/my-container/')<br>COMMENT = 'Azure Blob integration for Snowpipe';<br><br>2 Create Pipe with Event Grid<br>CREATE OR REPLACE PIPE my_azure_pipe<br>AUTO_INGEST = TRUE<br>AZURE_EVENT_GRID_SUBSCRIPTION = 'https://<region>.eventgrid.azure.net/api/events'<br>AS<br>COPY INTO my_table<br>FROM @azure_stage<br>FILE_FORMAT = (FORMAT_NAME = 'my_csv_fmt'); |

| Google Cloud Storage Integration & Pipe | Monitoring & Metadata |
|---|---|
| CREATE OR REPLACE STORAGE INTEGRATION gcp_int<br>TYPE = EXTERNAL_STAGE<br>STORAGE_PROVIDER = GCS<br>ENABLED = TRUE<br>STORAGE_ALLOWED_LOCATIONS = ('gcs://my-bucket/data/')<br>STORAGE_GCP_SERVICE_ACCOUNT = 'service-account@project.iam.gserviceaccount.com'<br>COMMENT = 'GCS integration for Snowpipe';<br><br>4.2 Create Pipe with Pub/Sub<br>CREATE OR REPLACE PIPE my_gcp_pipe<br>AUTO_INGEST = TRUE<br>GCP_PUBSUB_TOPIC = 'projects/my-project/topics/my-topic'<br>AS<br>COPY INTO my_table<br>FROM @gcp_stage<br>FILE_FORMAT = (TYPE = 'PARQUET'); | Query pipe definitions across account<br><br>SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.PIPES<br>WHERE PIPE_NAME LIKE '%pipe%';<br><br>-- Check pipe refresh history<br><br>SELECT * FROM TABLE(INFORMATION_SCHEMA.PIPE_USAGE_HISTORY(<br>START_TIME => DATEADD('day', -7, CURRENT_TIMESTAMP()),<br>END_TIME => CURRENT_TIMESTAMP()<br>)); |

| PUT (Local → Stage) | 2. GET (Stage → Local) | 3. COPY INTO (Stage → Table) |
|---|---|---|
| PUT file://<local_path> @<stage_name><br>[ AUTO_COMPRESS = { TRUE \| FALSE } ] -- gzip-compress during upload<br>[ OVERWRITE = { TRUE \| FALSE } ] -- replace existing staged file:content<br>[ PARALLEL = <integer> ] -- number of upload threads (Snowpark API)<br>[ RECURSIVE = TRUE \| FALSE ] -- include subdirectories (SnowSQL CLI)<br>[ PATTERN = '<regex>' ] -- upload only matching files<br>; | GET @<stage_name>/<path_pattern> 'file://<local_dir>/'<br>[ PARALLEL = <integer> ] -- threads in Snowpark<br>API:contentReference[oaicite:8]{index=8}<br>[ PATTERN = '<regex>' ] -- filter staged files<br>; | COPY INTO <table_name><br>FROM @<stage_name>/<prefix><br>[ FILE_FORMAT = (<br>FORMAT_NAME = '<fmt>' -- reference named format<br>\| TYPE = { CSV \| JSON \| PARQUET \| ... }<br>[ <format_options> ] -- inline options (see below)<br>) ]<br>[ PATTERN = '<regex>' ] -- select specific files<br>[ ON_ERROR = {<br>ABORT_STATEMENT -- default<br>\| CONTINUE -- skip bad rows<br>\| SKIP_FILE -- skip offending file<br>\| SKIP_FILE_<n> -- skip after n errors<br>} ]<br>[ VALIDATION_MODE = { RETURN_ERRORS \| RETURN_ALL_ERRORS } ]<br>[ PURGE = TRUE \| FALSE ] -- delete files on success<br>[ FORCE = TRUE \| FALSE ] -- load even if already loaded<br>; |

| 4. COPY INTO (Table → Stage) | 5. Listing & Inspecting |
|---|---|
| COPY INTO @<stage_name>/<path_prefix><br>FROM <table_name><br>[ FILE_FORMAT = (<br>TYPE = { CSV \| PARQUET \| JSON \| XML }<br>\| FORMAT_NAME = '<fmt>'<br>[ <format_options> ]<br>) ]<br>[ HEADER = TRUE \| FALSE ] -- include header (CSV)<br>[ SINGLE = TRUE \| FALSE ] -- single output file<br>[ MAX_FILE_SIZE = <bytes> ] -- target size for partitioned files<br>[ OVERWRITE = TRUE \| FALSE ]<br>[ COMPRESSION = { AUTO \| GZIP \| BZ2 \| BROTLI \| ZSTD \| DEFLATE } ]<br>; | sql<br>-- List staged files<br>LIST @<stage_name>/<path_prefix>;<br><br>-- Show all COPY history for a table<br>SELECT *<br>FROM INFORMATION_SCHEMA.LOAD_HISTORY<br>WHERE TABLE_NAME = '<table_name>'<br>AND LOADED_ON > DATEADD('day', -7, CURRENT_TIMESTAMP());<br><br>-- Check file export history<br>SELECT *<br>FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()))<br>WHERE ROW_NUMBER() > 0; -- after COPY INTO unload |

WAREHOUSE

| Create Warehouse | Alter Warehouse | START / STOP / SUSPEND / RESUME | SHOW & DESCRIBE and Account usage |
|---|---|---|---|
| CREATE OR REPLACE WAREHOUSE my_wh<br>WITH<br>WAREHOUSE_SIZE = 'LARGE' -- XS to 6XL<br>MIN_CLUSTER_COUNT = 1 -- minimum for multi-cluster<br>MAX_CLUSTER_COUNT = 3 -- maximum clusters for scaling<br>MAX_CONCURRENCY_LEVEL = 10 -- concurrent queries per cluster<br>AUTO_SUSPEND = 300 -- seconds of inactivity before suspend<br>AUTO_RESUME = TRUE -- auto-resume on query<br>INITIALLY_SUSPENDED = TRUE -- start in suspended state<br>RESOURCE_MONITOR = my_rm; -- attach resource monitor | ALTER WAREHOUSE my_wh<br>SET<br>WAREHOUSE_SIZE = 'X-LARGE' -- resize vertically<br>MIN_CLUSTER_COUNT = 2 -- adjust cluster bounds<br>MAX_CLUSTER_COUNT = 5 -- adjust cluster bounds<br>MAX_CONCURRENCY_LEVEL = 20 -- raise concurrency limit<br>AUTO_SUSPEND = 600 -- change suspend timeout<br>AUTO_RESUME = FALSE; -- disable auto-resume if needed | .-- Start a suspended warehouse<br>START WAREHOUSE my_wh;<br><br>-- Suspend a running warehouse immediately<br>SUSPEND WAREHOUSE my_wh;<br><br>-- Stop is synonymous with SUSPEND<br>STOP WAREHOUSE my_wh;<br><br>-- Resume is synonymous with START<br>RESUME WAREHOUSE my_wh; | .-- List all warehouses with current state and size<br>SHOW WAREHOUSES;<br><br>-- Describe a specific warehouse's detailed configuration<br>DESCRIBE WAREHOUSE my_wh;<br><br>SELECT<br>WAREHOUSE_NAME, USAGE_DATE, USAGE_HOUR, CREDITS_USED,<br>CREDITS_ATTRIBUTED_COMPUTE_QUERIES<br>FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY<br>WHERE WAREHOUSE_NAME = 'MY_WH'; |

Clone and timetravel

| Time Travel Queries - Query Historical Data | Restore Dropped Objects | Zero-Copy Cloning |
|---|---|---|
| .-- As of a specific timestamp (TIMESTAMP must be string or cast)<br>SELECT * FROM my_table<br>AT (TIMESTAMP => '2025-06-30 12:00:00'::TIMESTAMP_NTZ);<br>Supports TIMESTAMP, TIMESTAMP_LTZ, TIMESTAMP_TZ<br><br>-- As of an offset (seconds) before now<br>SELECT * FROM my_table<br>AT (OFFSET => -60*60*24); -- 24 hours ago OFFSET is in seconds<br><br>-- As of a specific statement (use QUERY_ID)<br>SELECT * FROM my_table<br>AT (STATEMENT => '01a1b2c3-...'); | .-- Restore the most recent dropped table named my_table<br>UNDROP TABLE my_table;<br>Restores within Time Travel retention (default 1 day, configurable up to 90)<br><br>-- Restore a dropped schema<br>UNDROP SCHEMA my_schema;<br><br>-- Restore a dropped database<br>UNDROP DATABASE my_database;<br><br>-- Restore a specific version by its table_id<br>UNDROP TABLE IDENTIFIER(<br>(SELECT table_id FROM my_db.INFORMATION_SCHEMA.TABLES<br>WHERE table_name='MY_TABLE' AND deleted IS NOT NULL<br>ORDER BY deleted DESC LIMIT 1)<br>); | 2.1 Clone a Table<br>-- Clone a table at current state<br>CREATE TABLE my_table_clone CLONE my_table;<br>-- Clone as of historical timestamp<br>CREATE TABLE my_table_clone<br>CLONE my_table AT (TIMESTAMP => '2025-06-30 12:00:00'); --CLONE supports AT and BEFORE<br><br>2.2 Clone a Schema<br>CREATE SCHEMA my_schema_clone<br>CLONE my_schema; --Clones all objects in the schema<br>-- Historical schema clone<br>CREATE SCHEMA hist_schema_clone<br>CLONE my_schema BEFORE (TIMESTAMP => DATEADD(day, -1, CURRENT_TIMESTAMP()));<br>-- BEFORE is alias for AT with TIMESTAMP<br><br>2.3 Clone a Database<br>CREATE DATABASE my_db_clone CLONE my_db; --Replica of entire database<br>-- Historical database clone<br>CREATE DATABASE hist_db_clone<br>CLONE my_db AT (OFFSET => -86400); -- 24h ago Supports OFFSET<br><br>2.4 Clone Other Objects<br>CREATE STREAM my_stream_clone CLONE my_stream;<br>CREATE STAGE my_stage_clone CLONE my_stage;<br>CREATE FILE FORMAT my_fmt_clone CLONE my_fmt; |

Stream and stream

| Create Stream | Drop, Show, Describe | Task syntax |
|---|---|---|
| CREATE [ OR REPLACE ] STREAM <stream_name><br>[ APPEND_ONLY = { TRUE \| FALSE } ] -- track apend only if TRUE<br>[ INSERT_ONLY = { TRUE \| FALSE } ] -- track inserts only (alias)<br>ON { TABLE \| VIEW } <db>.<schema>.<table_name><br>[ SHOW_INITIAL_ROWS = { TRUE \| FALSE } ] -- include existing rows on creation<br>[ COMMENT = '<string>' ]; | DROP STREAM IF EXISTS <stream_name>;<br>SHOW STREAMS IN SCHEMA <db>.<schema>;<br>DESC STREAM <db>.<schema>.<stream_name>;<br>-- In-database INFO_SCHEMA<br>SELECT * FROM <db>.INFORMATION_SCHEMA.STREAMS<br>WHERE STREAM_NAME = '<stream_name>';<br>-- Account-level usage<br>SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.STREAMS<br>WHERE STREAM_NAME = '<stream_name>'; | CREATE [ OR REPLACE ] TASK <task_name><br>[ WAREHOUSE = <wh_name> ]<br>[ SCHEDULE = '<cron_expression>' -- cron syntax<br>\| SCHEDULE = 'USING CRON <expr> <tz>' ]<br>[ SCHEDULE = '<n> MINUTE' ] -- every n minutes<br>[ AFTER <parent_task1>[, <parent_task2> ...] ] -- run after successful parent(s)<br>[ WHEN SYSTEM$STREAM_HAS_DATA('<stream_name>') ] -- triggered on CDC<br>[ USER_TASK_TIMEOUT_MS = <integer> ] -- max run time<br>[ COMMENT = '<string>' ]<br>AS<br><sql_statement>; |

| Alter task | Resume, suspend and details related task |
|---|---|
| ALTER TASK <task_name><br>SET WAREHOUSE = <new_wh>,<br>SCHEDULE = '2 HOUR',<br>USER_TASK_TIMEOUT_MS = 600000;<br><br>ALTER TASK <task_name><br>MODIFY WHEN SYSTEM$STREAM_HAS_DATA('other_stream');<br><br>ALTER TASK <task_name><br>ADD AFTER another_task;<br><br>ALTER TASK <task_name><br>REMOVE AFTER old_task; | SHOW TASKS IN SCHEMA <db>.<schema>;<br>DESC TASK <db>.<schema>.<task_name>;<br>DROP TASK IF EXISTS <task_name>;<br><br>ALTER TASK <task_name> RESUME;<br>ALTER TASK <task_name> SUSPEND;<br><br>-- INFORMATION_SCHEMA view<br>SELECT * FROM <db>.INFORMATION_SCHEMA.TASKS<br>WHERE TASK_NAME = '<task_name>';<br><br>-- Account_USAGE Task History<br>SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.TASK_HISTORY<br>WHERE TASK_NAME = '<task_name>'<br>AND START_TIME > DATEADD('day', -7, CURRENT_TIMESTAMP()); |

Stored Procedures and UDF (SQL, JAVA, Python, Scala, )

| Create Procedure in SQL and JAVA () | Alter / Drop / Show / Describe | User-Defined Functions | Alter / Drop / Show / Describe2 |
|---|---|---|---|
| 1.1 Create Procedure (SQL Scripting)<br>CREATE OR REPLACE PROCEDURE output_message(msg VARCHAR)<br>RETURNS VARCHAR NOT NULL<br>LANGUAGE SQL<br>COMMENT = 'Echoes the input message'<br>AS<br>$$<br>BEGIN<br>RETURN msg;<br>END;<br>$$;<br><br>1.2 Create Procedure (JavaScript)<br>CREATE OR REPLACE PROCEDURE js_insert_count(tbl_name STRING)<br>RETURNS VARCHAR<br>LANGUAGE JAVASCRIPT<br>RUNTIME_VERSION = '1.8'<br>HANDLER = 'run'<br>AS<br>$$<br>function run(session, tbl_name) {<br>const result = session.sql(`INSERT INTO log_table SELECT * FROM ${tbl_name}`).execute();<br>return `Inserted ${result.getRowCount()} rows`;<br>}<br>$$; | Alter / Drop / Show / Describe<br><br>ALTER PROCEDURE output_message COMMENT = 'Updated comment';<br>SHOW PROCEDURES [IN SCHEMA my_db.my_schema];<br>DESC PROCEDURE js_insert_count;<br>DROP PROCEDURE IF EXISTS scala_proc;<br><br>--Call & Exceptions<br>CALL output_message('Hi Snowflake');  -- returns 'Hi Snowflake'<br>CALL js_insert_count('SRC');  -- may throw SQLCompilationError if table missing | 2.1 Create Scalar SQL UDF<br>CREATE OR REPLACE FUNCTION add_numbers(x INT, y INT)<br>RETURNS INT<br>LANGUAGE SQL<br>COMMENT = 'Adds two integers'<br>AS<br>$$<br>x + y<br>$$;<br><br>2.2 Create Tabular SQL UDF (UDTF)<br>CREATE OR REPLACE FUNCTION split_csv(str STRING)<br>RETURNS TABLE (item STRING)<br>LANGUAGE SQL<br>AS<br>$$<br>SELECT value AS item<br>FROM TABLE(FLATTEN(INPUT => SPLIT(str, ',')));<br>$$;<br><br>2.3 Create UDF with JavaScript<br>CREATE OR REPLACE FUNCTION js_len(str STRING)<br>RETURNS INT<br>LANGUAGE JAVASCRIPT<br>AS<br>$$<br>return str.length;<br>$$; | ALTER FUNCTION add_numbers SET COMMENT = 'Sum of integers';<br>SHOW USER FUNCTIONS IN SCHEMA my_db.my_schema;<br>DESC FUNCTION split_csv;<br>DROP FUNCTION IF EXISTS js_len; |

User, role, Grant, privilage

| Create User | Role | Grant Role to User or Role | Privilege Management |
|---|---|---|---|
| CREATE USER <user_name><br>PASSWORD = '<string>'  -- required unless EXTERNAL_OAUTH<br>[ LOGIN_NAME = '<string>' ]  -- defaults to USER_NAME<br>[ DEFAULT_ROLE = '<role>' ]  -- initial active role<br>[ DEFAULT_WAREHOUSE = '<warehouse>' ]  -- auto-resume warehouse<br>[ DEFAULT_NAMESPACE = '<db>.<schema>' ]  -- default schema context<br>[ MUST_CHANGE_PASSWORD = TRUE \| FALSE ]  -- force reset on first login<br>[ DISABLED = TRUE \| FALSE ]  -- lock account<br>[ COMMENT = '<string>' ];<br><br>Alter User-<br>ALTER USER <user_name><br>SET PASSWORD = '<new_password>'<br>[ SET DEFAULT_ROLE = '<role>' ]<br>[ SET DEFAULT_WAREHOUSE = '<warehouse>' ]<br>[ SET DEFAULT_NAMESPACE = '<db>.<schema>' ]<br>[ SET DISABLED = TRUE \| FALSE ]<br>[ SET MUST_CHANGE_PASSWORD = TRUE \| FALSE ];<br><br>DROP USER IF EXISTS <user_name>;<br>SHOW USERS;<br>DESCRIBE USER <user_name>; | 1. Create Role:<br>CREATE ROLE <role_name><br>[ COMMENT = '<string>' ];<br>-- or CREATE OR REPLACE ROLE <role_name>;<br><br>2 Alter Role:<br>ALTER ROLE <role_name><br>SET COMMENT = '<new_comment>';<br><br>3 Drop & List Roles:<br>DROP ROLE IF EXISTS <role_name>;<br>SHOW ROLES;<br>DESC ROLE <role_name>;<br><br>4 Role Hierarchies:<br>-- Grant a child role to a parent role<br>GRANT ROLE <child_role> TO ROLE <parent_role>; | 1. Grant Role to User or Role<br>GRANT ROLE <role_name><br>TO USER <user_name>;<br><br>GRANT ROLE <role_name><br>TO ROLE <another_role>;<br><br>2. Revoke Role from User or Role<br><br>REVOKE ROLE <role_name><br>FROM USER <user_name>;<br><br>REVOKE ROLE <role_name><br>FROM ROLE <another_role>;<br><br>-- What roles does user have?<br>SHOW GRANTS TO USER <user_name>;<br><br>-- What privileges does role have?<br>SHOW GRANTS TO ROLE <role_name>;<br><br>-- Account-wide audit<br>SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.GRANTS_TO_ROLES<br>WHERE ROLE = '<role_name>'; | 4.1 Grant Privilege on Object to Role<br>GRANT <privilege>[, ...]<br>ON { TABLE \| VIEW \| SCHEMA \| DATABASE \| WAREHOUSE \| ... } <object_name><br>TO ROLE <role_name><br>[ WITH GRANT OPTION ];<br><br>4.2 Revoke Privilege on Object from Role<br>REVOKE <privilege>[, ...]<br>ON { TABLE \| VIEW \| SCHEMA \| DATABASE \| WAREHOUSE \| ... } <object_name><br>FROM ROLE <role_name>;<br><br>4.3 Grant Privilege on Object to User<br>GRANT <privilege><br>ON <object><br>TO USER <user_name>;<br><br>4.4 Revoke Privilege from User<br>REVOKE <privilege><br>ON <object><br>FROM USER <user_name>;<br><br>4.5 Global & Account-Level Privilege<br>-- Global<br>GRANT MANAGE GRANTS TO ROLE securityadmin;<br>-- Account<br>GRANT CREATE DATABASE ON ACCOUNT TO ROLE dataeng; |